

C++ Standard Proposal — A Civil-Time Library

Document number:	P0215R0
Date:	2016-02-11
Reply-to:	Greg Miller (igm@google.com), Bradley White (bww@google.com)
Audience:	Library Evolution Working Group

Table of Contents

- [Introduction](#)
- [Motivation and Scope](#)
- [The Conceptual Model and Terminology](#)
- [Impact on the Standard](#)
- [Design Decisions](#)
- [Technical Specification](#)
- [Synopsis](#)
- [Examples](#)
- [References](#)

Introduction

This document proposes a standard C++ library for computing with time that is represented by years, months, and days that follow the rules of the [Proleptic Gregorian Calendar](#), as well as 24-hour days that are divided into hours, minutes, and seconds. These six fields are how humans commonly experience time, and this time-zone-independent representation is called *civil time*.

Motivation and Scope

Programming with time on a human scale is notoriously difficult and error prone: time zones are complicated, daylight-saving time (DST) is complicated, calendars are complicated, and leap seconds are complicated. These complexities quickly surface in code because programmers do not have a simple conceptual model with which to reason about the time-programming challenges they are facing. Furthermore, this lack of a simple conceptual model begets the lack of a simple library, leaving only complicated libraries that programmers struggle to understand and use correctly (or no libraries at all).

A survey of application-level code within Google that computes with civil-time fields shows that programmer confusion is a fundamental cause of overly complicated and buggy code. This is a result of the inherent complexities in this problem space as just mentioned, and the fact that there has been little progress toward hiding these complexities behind a library that presents a simpler conceptual model to programmers.

The Civil-Time Library proposed here addresses these problems by presenting civil time as a *regular*, time-zone-independent concept that allows programmers to ignore the above complexities when it is safe to do so. This library has been implemented and widely used within Google for the past few years. It is actively being used in real-world code by novice and expert programmers alike.

The Conceptual Model and Terminology

Civil time, as described in this proposal, is just one of three concepts that comprise a complete, straightforward, and language-neutral conceptual model for reasoning about time programming: *absolute time*, *civil time*, and *time zone*.

Absolute time uniquely and universally represents a specific instant in time. It has no notion of calendars, or dates, or times of day. Instead, it is a measure of the passage of real time, typically as a simple count of ticks since some epoch. Absolute times are independent of all time zones and do not suffer from human-imposed complexities such as daylight-saving time (DST). Many C++ types exist to represent absolute times, classically `time_t` and more recently `std::chrono::time_point`.

Civil time is the legally recognized representation of time for ordinary affairs (cf. <http://www.merriam-webster.com/dictionary/civil>). It is a human-scale representation of time that consists of the six fields — year,

month, day, hour, minute, and second (sometimes shortened to "YMDHMS") — and it follows the rules of the [Proleptic Gregorian Calendar](#), with 24-hour days divided into 60-minute hours and 60-second minutes. Like absolute times, civil times are also independent of all time zones and their related complexities (e.g., DST). While `std::tm` contains the six civil-time fields (YMDHMS), plus a few more, it does not have behavior to enforce the rules of civil time.

Time zones are geo-political regions within which human-defined rules are shared to convert between absolute-time and civil-time domains. A time-zone's rules include things like the region's offset from the [UTC](#) time standard, daylight-saving adjustments, and short abbreviation strings. Time zones often have a history of disparate rules that apply only for certain periods, because the rules may change at the whim of a region's local government. For this reason, time-zone rules are usually compiled into data snapshots that are used at runtime to perform conversions between absolute and civil times. There is currently no C++ standard library supporting arbitrary time zones.

The C++ standard already has the `<chrono>` library, which is a good implementation of an *absolute-time* library. This paper proposes a standard *civil-time* library. A separate paper ([P0216R0](#)) proposes a standard *Time-Zone* Library that bridges `<chrono>` and this Civil-Time Library, and will complete the three pillars of the conceptual model just described.

Impact on the Standard

The Civil-Time Library proposed here does not directly depend on any other standard C++ library. It is implementable using C++98, with optimizations such as `constexpr` available when compiled as C++11.

The current sister proposal for a standard Time-Zone Library ([P0216R0](#)) depends on the existence of this Civil-Time Library.

Design Decisions

Only the Proleptic Gregorian Calendar is used

Many different calendaring systems exist in the world, but the most commonly used calendar is the [Proleptic Gregorian Calendar](#). Supporting rarely used calendars would add complexity to the library with little value. Additionally, international standards such as [UTC](#) rely on the Gregorian Calendar. Therefore, we've chosen not to support other calendars.

Leap seconds are disregarded

Unlike leap days, which are part of the [Proleptic Gregorian Calendar](#), leap seconds are unpredictable and would require time zone knowledge to properly support their use in the civil-time domain. Such time zone awareness would add substantial conceptual complexity for a rare use case. The simplicity of this Civil-Time Library stems from presenting a regularized, or idealized, human-scale time representation that is devoid of complexities and discontinuities, such as those caused by supporting DST and leap seconds. We believe it would be a mistake to expose leap second complexities in the civil-time domain. And if knowledge of leap seconds were necessary, that support should exist in a separate time zone library rather than a civil-time library.

Civil times are always valid

It is not possible to create an invalid civil-time instance. Similarly, there are no errors when constructing a civil time with field values that are out of range. This is enforced by normalizing input fields (similar to `mktime(3)`). For example, January 32 will normalize to February 1. This decision reduces the amount of boilerplate error checking, and it allows callers to do arithmetic on input field arguments without worrying about range. If normalization is undesired, callers may compare the resulting normalized fields to the input fields to signal an error.

Civil times do not have subseconds

Civil times are represented by the six fields of year, month, day, hour, minute, and second. These are the six fields that are typically considered for human-scale time. Additionally, seconds are the level of precision within time zone data files; i.e., there are no subsecond time zone transitions. While it would be possible to support subseconds in this Civil-Time Library, doing so would complicate the library for a feature with little demand.

Civil times are aligned to a civil-field boundary

One of the classic questions that arises when talking about a civil-time library (or a date library or a date/time library) is this: "What happens when you add a month to January 31?" This is an interesting question because there could be a number of possible answers:

- March 3 (or 2 if a leap year). This may make sense if the operation wants the equivalent of February 31.
- February 28 (or 29 if a leap year). This may make sense if the operation wants the last day of January to go to the last day of February.
- Error. The caller may get some error, an exception, an invalid date object, or maybe `false` is returned. This may make sense because there is no single unambiguously correct answer to the question.

Practically speaking, any answer that is not what the programmer intended is the wrong answer.

The Civil-Time Library proposed here avoids this problem by making it impossible to ask such an ambiguous question. All civil-time objects are aligned to a particular civil-field boundary (such as aligned to a year, month, day, hour, minute, or second), and arithmetic operates on the field to which the object is aligned. This means that in order to "add a month" the object must first be aligned to a month boundary, which is equivalent to the first day of that month. See the [Technical Specification](#) section below for more about alignment.

There are indeed ways to accomplish the task of adding a month to January 31 with this Civil-Time Library, but they require the programmer to be more explicit about their intent so the answer is unsurprising. There is an example showing this later in this paper. In practice, we have found few places in Google code where programmers wanted unaligned date arithmetic, and in those few cases the more explicit code helped ensure that the programmer got the answer they expected.

Technical Specification

The core of the Civil-Time Library is based on the following six individual classes:

- `civil_second`
- `civil_minute`
- `civil_hour`
- `civil_day`
- `civil_month`
- `civil_year`

Each class is a simple value type with the same interface for construction and the same six accessors for each of the YMDHMS fields. These classes differ only in their *alignment*, which is indicated by the type name and specifies the field on which arithmetic operates.

Each class can be constructed by passing up to six optional integer arguments representing the YMDHMS fields (in that order) to the constructor. Omitted fields are assigned their minimum valid value. Hours, minutes, and seconds will be set to 0, month and day will be set to 1, and since there is no minimum valid year it will be set to 1970. So, a default-constructed civil-time object will have YMDHMS fields representing "1970-01-01 00:00:00".

Each civil-time class is aligned to the civil-time field indicated in the class's name. Alignment is performed by setting all the inferior fields to their minimum valid value (as described above). The following are examples of how each of the six types would align the fields representing November 22, 2015 at 12:34:56 in the afternoon. (Note: the string format used here is not important; it's just a shorthand way of showing the six YMDHMS fields.)

Class	Example alignment
<code>civil_second</code>	2015-11-22 12:34:56
<code>civil_minute</code>	2015-11-22 12:34:00
<code>civil_hour</code>	2015-11-22 12:00:00
<code>civil_day</code>	2015-11-22 00:00:00
<code>civil_month</code>	2015-11-01 00:00:00
<code>civil_year</code>	2015-01-01 00:00:00

Each civil-time type performs arithmetic on the field to which it is aligned. This means that adding 1 to a `civil_day` increments the day field (normalizing as necessary), and subtracting 7 from a `civil_month` operates on the month field (normalizing as necessary). All arithmetic produces a new value that represents a valid civil time. Difference requires two similarly aligned civil time types and returns the scalar answer in units of the given alignment. For example, the difference between two `civil_hour` objects will give an answer in hours.

Finally, in addition to the six civil-time types just described, there are a handful of helper functions and algorithms for performing common calculations. These are described in the [Synopsis](#) below.

Synopsis

The following code illustrates the public API for each of the six civil-time types described above. As an implementation choice, Google chose to write one class template that is parameterized on the alignment field as a tag struct. This class template is not a public API point, but it serves here to illustrate the API of each of the public civil-time types.

```
namespace detail {
template <typename T>
class civil_time {
public:
    explicit constexpr civil_time(int y, int m = 1, int d = 1,
                                  int hh = 0, int mm = 0, int ss = 0);

    constexpr civil_time() : civil_time(1970) {}
    constexpr civil_time(const civil_time&) = default;
    civil_time& operator=(const civil_time&) = default;

    // Explicit conversion between civil times of different alignment.
    template <typename U>
    explicit constexpr civil_time(civil_time<U>);

    // Field accessors.
    constexpr int year() const;
    constexpr int month() const;
    constexpr int day() const;
    constexpr int hour() const;
    constexpr int minute() const;
    constexpr int second() const;

    // Assigning arithmetic.
    civil_time& operator+=(int);
    civil_time& operator-=(int);
    civil_time& operator++();
    civil_time operator++(int);
    civil_time& operator--();
    civil_time operator--(int);

    // Binary arithmetic operators.
    inline friend constexpr civil_time operator+(const civil_time&, int) { ... }
    inline friend constexpr civil_time operator+(int, const civil_time&) { ... }
    inline friend constexpr civil_time operator-(const civil_time&, int) { ... }
    inline friend constexpr int operator-(const civil_time&, const civil_time&) { ... }

private:
    ....
};

// Relational operators that work with differently aligned objects.
// Always compares all six YMDHMS fields.
template <typename T1, typename T2>
constexpr bool operator<(const civil_time<T1>&, const civil_time<T2>&);
template <typename T1, typename T2>
constexpr bool operator<=(const civil_time<T1>&, const civil_time<T2>&);
template <typename T1, typename T2>
constexpr bool operator>=(const civil_time<T1>&, const civil_time<T2>&);
template <typename T1, typename T2>
constexpr bool operator>(const civil_time<T1>&, const civil_time<T2>&);
template <typename T1, typename T2>
constexpr bool operator==(const civil_time<T1>&, const civil_time<T2>&);
template <typename T1, typename T2>
constexpr bool operator!=(const civil_time<T1>&, const civil_time<T2>&);

struct year_tag {};
struct month_tag {};
struct day_tag {};
struct hour_tag {};
struct minute_tag {};
struct second_tag {};
} // namespace detail

// The six public civil-time types.
```

```

using civil_year = detail::civil_time<detail::year_tag>;
using civil_month = detail::civil_time<detail::month_tag>;
using civil_day = detail::civil_time<detail::day_tag>;
using civil_hour = detail::civil_time<detail::hour_tag>;
using civil_minute = detail::civil_time<detail::minute_tag>;
using civil_second = detail::civil_time<detail::second_tag>;

```

In addition to the six civil-time types defined above, the following helper functions provide help with common computations.

```

enum class weekday {
    monday,
    tuesday,
    wednesday,
    thursday,
    friday,
    saturday,
    sunday,
};

// Returns the weekday of the argument.
constexpr weekday get_weekday(const civil_day&);

// Returns the civil_day that strictly follows/precedes the argument and that
// falls on the given weekday.
constexpr civil_day next_weekday(const civil_day&, weekday);
constexpr civil_day prev_weekday(const civil_day&, weekday);

// Returns the yearday of a civil_day.
constexpr int get_yearday(const civil_day&);

```

Examples

The following examples show usage of the proposed Civil-Time Library, as well as illustrate interesting semantics of various API points.

As a shorthand in the examples below, the YMDHMS fields of a civil-time type may be shown as a string of the form YYYY-MM-DD hh:mm:ss. Fields omitted from the string format still exist and are assumed to have their minimum value. For example:

```

civil_day d(2015, 2, 3);
// d.year() == 2015
// d.month() == 2
// d.day() == 3
// d.hour() == 0
// d.minute() == 0
// d.second() == 0
// Shorthand: d == 2015-02-03
// ...same as d == 2015-02-03 00:00:00

```

Construction

Each of the civil-time types can be constructed in two ways: by directly passing up to six (optional) integers representing YMDHMS fields to the constructor, or by copying the fields from a differently aligned civil-time type.

```

civil_day default_value; // 1970-01-01 00:00:00

civil_day a(2015, 2, 3); // 2015-02-03 00:00:00
civil_day b(2015, 2, 3, 4, 5, 6); // 2015-02-03 00:00:00
civil_day c(2015); // 2015-01-01 00:00:00

civil_second ss(2015, 2, 3, 4, 5, 6); // 2015-02-03 04:05:06
civil_minute mm(ss); // 2015-02-03 04:05:00
civil_hour hh(mm); // 2015-02-03 04:00:00
civil_day d(hh); // 2015-02-03 00:00:00
civil_month m(d); // 2015-02-01 00:00:00
civil_year y(m); // 2015-01-01 00:00:00

m = civil_month(y); // 2015-01-01 00:00:00

```

```
d = civil_day(m);           // 2015-01-01 00:00:00
hh = civil_hour(d);         // 2015-01-01 00:00:00
mm = civil_minute(hh);      // 2015-01-01 00:00:00
ss = civil_second(mm);      // 2015-01-01 00:00:00
```

Comparison

Comparison always considers all six YMDHMS fields, regardless of the type's alignment. Comparison between differently aligned civil-time types is allowed.

Note that comparison between differently aligned types is not a critical part of this API. It exists as a convenience, but it could be removed at the cost of requiring explicit casts between civil-time types when comparing.

```
civil_day feb_3(2015, 2, 3); // 2015-02-03 00:00:00
civil_day mar_4(2015, 3, 4); // 2015-03-04 00:00:00
// feb_3 < mar_4
// civil_year(feb_3) == civil_year(mar_4)

civil_second feb_3_noon(2015, 2, 3, 12, 0, 0); // 2015-02-03 12:00:00
// feb_3 < feb_3_noon
// feb_3 == civil_day(feb_3_noon)

// Iterates all the days of February 2015.
for (civil_day d(2015, 2, 1); d < civil_month(2015, 3); ++d) {
    // ...
}
```

Arithmetic

Civil-time types support natural arithmetic operators such as addition, subtraction, and difference. Arithmetic operates on the civil-time field indicated in the type's name. Difference requires arguments with the same alignment and returns the answer in units of the alignment.

```
civil_day a(2015, 2, 3);
++a;           // 2015-02-04 00:00:00
--a;           // 2015-02-03 00:00:00
civil_day b = a + 1; // 2015-02-04 00:00:00
civil_day c = 1 + b; // 2015-02-05 00:00:00
int n = c - a;      // n = 2 (days)
int m = c - civil_month(c); // Won't compile: different types.
```

Weekdays

The Civil-Time Library provides the `prev_weekday()` and `next_weekday()` functions for navigating the calendar by the day of the week. Both functions accept a `civil_day` argument as well as the desired `weekday`. They both return a strictly different `civil_day` that falls on the given `weekday`, even if the argument was already on the requested `weekday`.

```
//      August 2015
// Su Mo Tu We Th Fr Sa
//              1
//  2  3  4  5  6  7  8
//  9 10 11 12 13 14 15
// 16 17 18 19 20 21 22
// 23 24 25 26 27 28 29
// 30 31

civil_day a(2015, 8, 13); // get_weekday(a) == weekday::thursday
civil_day b = next_weekday(a, weekday::thursday); // 2015-08-20
civil_day c = prev_weekday(a, weekday::thursday); // 2015-08-06

civil_day d = ...
// Gets the following Thursday if d is not already Thursday
civil_day ceil_thursday = prev_weekday(d, weekday::thursday) + 7;
// Gets the previous Thursday if d is not already Thursday
civil_day floor_thursday = next_weekday(d, weekday::thursday) - 7;
```

Adding a month to January 31

[Design Decisions](#) explained why adding a month to January 31 presents an ambiguous question. As we've described, aligned date arithmetic makes it impossible to ask this ambiguous question directly. However, if a programmer really wants to ask this question, they need to be more explicit about the answer they expect. For example:

```
const civil_day d(2015, 1, 31);

// Answer 1:
// Adds 1 to the month field in the constructor, and let normalization happen.
const auto ans_normalized = civil_day(d.year(), d.month() + 1, d.day());
// ans_normalized == 2015-03-03 (aka Feb 31)

// Answer 2:
// Adds 1 to month field, capping at the end of next month
const auto last_day_of_next_month = civil_day(civil_month(d) + 2) - 1;
const auto ans_capped = std::min(ans_normalized, last_day_of_next_month);
// ans_capped == 2015-02-28

// Answer 3:
if (civil_month(ans_normalized) - civil_month(d) != 1) {
    // error, month overflow
}
```

References

- https://en.wikipedia.org/wiki/Civil_time
- <http://www.merriam-webster.com/dictionary/civil>
- <http://www.timeanddate.com/time/aboututc.html>
- https://en.wikipedia.org/wiki/Coordinated_Universal_Time
- https://en.wikipedia.org/wiki/Proleptic_Gregorian_calendar
- <https://github.com/google/cctz>